

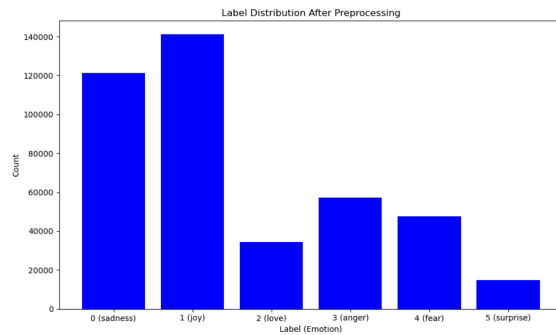
Alex Litchfield
Tomek Strzalkowski
Social Computing
16 April 2025

Homework 3 Report

In this homework I used an [english twitter dataset](#) that contains over 400,000 tweets and a classification label corresponding to emotions. The emotions are classified into 6 categories: sadness (0), joy (1), love (2), anger (3), fear (4), and surprise (5). The hypothesis was “Using the contents of a tweet, the primary emotion being conveyed by the user can be predicted”, which I do using Naïve Bayes.

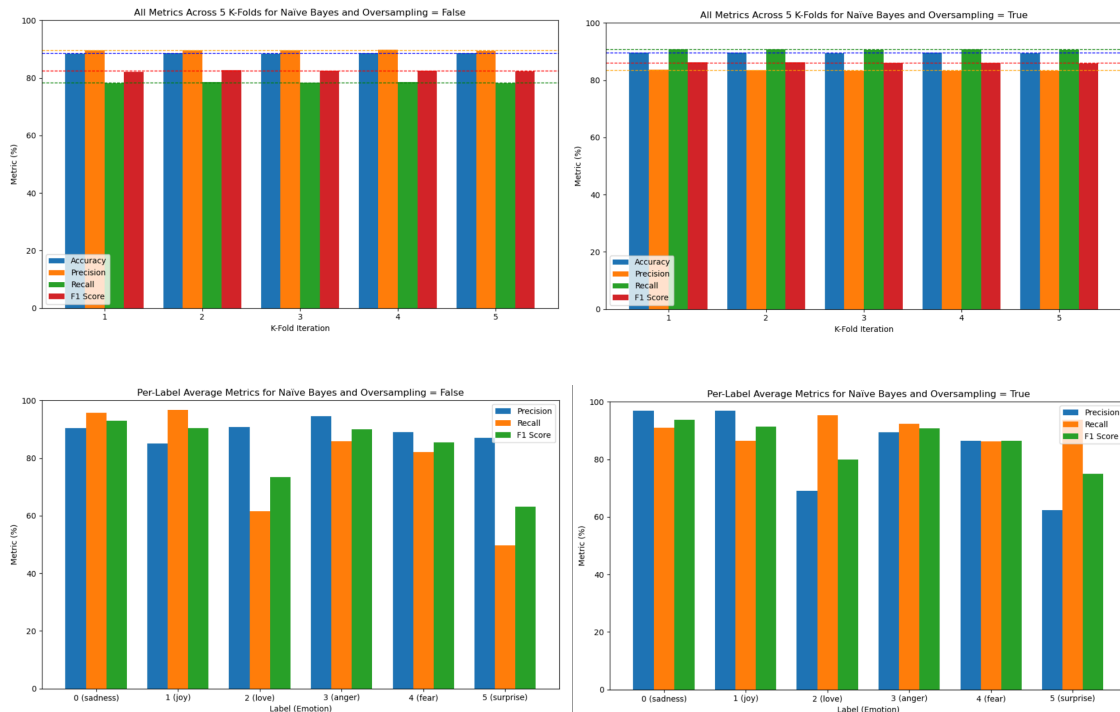
1. The features I chose to derive from the textual content of the data were unigrams (individual words), bigrams (word pairs), and TF-IDF (Term Frequency-Inverse Document Frequency). The textual content derived from the data were the tweets themselves. Unigrams were chosen as default to gloss over each word, and bigrams were added afterward as unigrams alone resulted in lower metrics. Certain words or word patterns could point to a prediction on the emotion being conveyed. TF-IDF was used to weight the importance of certain words in the data, effectively nullifying common words such as “and”, “is”, or “the”.
2. In homework 2 I used NB (Naïve Bayes) and SVM (Support Vector Machine) with a linear kernel. While both classifiers likely could have completed the current task, I found SVM to perform slightly worse and take 3 times as long as NB in the previous homework, so I decided to not use it for this assignment. NB is an appropriate choice for the current task since it is a classic text classification problem, which NB is good at. Since NB is a simple probabilistic classifier, it requires little parameters to build its models and is often faster than other existing algorithms. It also assumes that a feature is independent of the others, which is useful when trying to compute “total likeliness” of something as opposed to a percentage chance overall. Since we only extracted “label” and “text” from each user, the dataset was not overly complex, just rather large due to its number of entries. This made NB better suited as this classifier is faster and more efficient in this scenario. Only NB is used in the code since SVM took too long to train and yield any results.
3. K-fold cross validation involves the process of splitting the training set into k smaller sets, where during each iteration the current k set is used as the validation set and the remaining k-1 sets are used as the training set. I decided to use 5 fold cross validation, as anything below 5 results in worse end result metrics and anything higher did not significantly alter the metrics but increased the running time of the program. K fold cross validation was useful in this specific instance because it ensured that the model’s performance was not dependent on particular breakdowns of test/training data splits. K fold cross validation also reduced potential bias by training through multiple smaller sets. The metrics used to show the performance of the classifier were accuracy, precision, recall, and F1 score. The accuracy measured how many of the predictions were correct, the precision measured how many emotion labels provided by the model’s prediction were correct when compared with the true labels, and the recall measured how many of the emotion labels were actually correctly assigned when compared to the true labels. The F1 Score score measured how balanced the model was, as it combines precision and recall while also utilizing their harmonic mean. Therefore, a higher F1 score implies a higher precision and recall.
4. Implementation Information
 - a. The data preprocessing first involved creating a dataframe using `pandas.read_csv()` on the dataset that only kept the text content and labels of each entry. Entries were then removed if they were missing data in either the “text” or “label” columns. Stopword removal was then performed on the dataframe, removing common words like “and”, “is”, or “the”. The text was then converted to numbers through TF-IDF (tokenization). Aside from using the pandas library to create usable

dataframes, the sklearn module was used as well. To handle the preprocessing actions and feature extraction, sklearn.feature_extraction.text was used to get TfidfVectorizer(). In addition, sklearn.model_selection was used to get the KFold class, giving the ability to loop through k folds. As seen in the graph below, the label distribution within the dataset, even after preprocessing, was extremely uneven. This is why there are 2 versions of the model, one where this was corrected through oversampling and one where it was not.



- b. Using sklearn.feature_extraction.txt to get TfidfVectorizer(), feature extraction was performed on the data. This included tokenization and stopwords removal. The vocabulary was limited to 5000 of the most relevant words and applied unigrams and bigrams to determine word usage patterns in each of the labels. Vocabulary limits above or below 5000 resulted in lower scores due to not enough information or too many irrelevant words being tracked.
- c. The class MultinomialNB was retrieved from sklearn.naive_bayes to get the NB classifier, which trains on TF-IDF in numerical format. Normalization was not applied to the TF-IDF features since the TF-IDF processes already apply normalization to the given data by changing the term frequency based on how many times they occur in the dataset. Standardization was also not applied since the TF-IDF values were already in the range of 0 and 1.
- d. For this k fold cross validation, $k = 5$ was used to keep performance consistent over time. K values below 5 resulted in much lower metrics and K values above 5 offered minimal improvement while significantly increasing the model runtime. In each k fold, the training and testing sets are found by using `trainingSet = df.iloc[i]` and `testingSet = df.iloc[j]`, where i and j are the training and testing indexes of the current k fold, respectively. This selects the row that the fold will use from the dataframe in each set. Feature extraction is then applied through `vectorizer = TfidfVectorizer(max_features = 5000, ngram_range = (1, 2), stop_words = "english")`, `trainTFIDF = vectorizer.fit_transform(trainingSet["text"])`, and `testTFIDF = vectorizer.transform(testingSet["text"])`. Next, the function takes in whether or not the model should apply oversampling. This was done to show the label imbalances in the data if they were there and account for them. If oversampling is not applied, the training set is applied to the classifier through `classifier.fit(trainTFIDF, trainingSet["label"])`. If it is applied, a `RandomOverSampler()` is declared from `imblearn.over_sampling` followed by `X_resampled, y_resampled = ros.fit_resample(trainTFIDF, trainingSet["label"])` and `classifier.fit(X_resampled, y_resampled)`. This applies the newly oversampled trainData and trainTFIDF data to the classifier instead. The prediction is then created based off of the test data through `prediction = classifier.predict(testTFIDF)`, regardless of whether or not oversampling occurred. This information was then used to find the metrics for the current k fold. The libraries used to get these functions were pandas, TfidfVectorizer through sklearn.feature_extraction.text, and RandomOverSampler through imblearn.over_sampling.

- e. The metrics for performance evaluation of the classifier were accuracy, precision, recall, and F1 score which were found using `accuracy_score(testingSet["label"], prediction) * 100`, `precision_score(testingSet["label"], prediction, average='macro') * 100`, `recall_score(testingSet["label"], prediction, average='macro') * 100`, and `f1_score(testingSet["label"], prediction, average='macro') * 100` from `sklearn.metrics`. Each was multiplied by 100 at the end to be displayed as percentages later on. To get the average score for each metric, each metric's score in a given k fold was appended to a list which then was summed and divided by its length, giving the mean. To get the metrics for each label, `precision_recall_fscore_support` was used from `sklearn.metrics`. The output was printed in the jupyter notebook.
5. The performance metrics can be found in the jupyter notebook for `oversampling = True` or `False`.



Analysis: It is worth noting that for each dataset, the 4th k fold will typically have the highest score for each metric. This shows the need for the additional k folds, as without them, unseen data combinations would not have been used and would result in significantly higher average metrics across the board in all datasets. It can also be seen that accuracy, recall, and F1 score receive higher average scores on higher scores on each k fold when oversampling is used. While the precision scores do dip slightly in this case, the overall increase of the other metrics implies that the model performs slightly better when oversampling is used. The second set of graphs show the average precision, recall, and F1 score for each individual label over all k folds. When oversampling is used, all metrics increase for each label with the exception of the precision metric for the “surprise” label. However, since the majority of the scores improve, it is safe to again say that the model performs better when over sampling is used.